

Guía Completa para el Examen de Backend DeliverUS

Tabla de Contenidos

1. Introducción al Proyecto DeliverUS
 2. Conceptos Básicos de JavaScript para Backend
 3. Arquitectura del Proyecto Backend
 4. Migraciones de Base de Datos (Sequelize)
 5. Modelos (Models)
 6. Controladores (Controllers)
 7. Rutas (Routes)
 8. Middlewares
 9. Validaciones (express-validator)
 10. Ejemplos Completos Resueltos
 11. Checklist para el Examen
 12. Errores Comunes y Soluciones
-

1. Introducción al Proyecto DeliverUS

DeliverUS es una aplicación de delivery de comida similar a Glovo o Uber Eats. El backend está construido con:

- **Node.js:** Entorno de ejecución de JavaScript
- **Express:** Framework web para crear APIs REST
- **Sequelize:** ORM (Object-Relational Mapping) para manejar la base de datos
- **express-validator:** Librería para validar datos de entrada

Estructura típica del examen

En cada examen debes implementar una **nueva entidad** con sus operaciones CRUD (Create, Read, Update, Delete). Los pasos son siempre los mismos:

1. **Migraciones:** Crear/modificar tablas en la base de datos
 2. **Modelos:** Definir la estructura de datos y relaciones
 3. **Rutas:** Definir los endpoints de la API
 4. **Validaciones:** Validar los datos de entrada
 5. **Controladores:** Implementar la lógica de negocio
 6. **Middlewares:** Añadir comprobaciones de seguridad
-

2. Conceptos Básicos de JavaScript para Backend

2.1. Variables y Constantes

```
// const: valor inmutable (no se puede reasignar)
const PI = 3.14159
const userName = 'Juan'
```

```
// let: valor mutable (se puede reasignar)
let contador = 0
contador = contador + 1
```

```
// NO usar var (obsoleto)
```

2.2. Funciones

```
// Función tradicional
function sumar(a, b) {
  return a + b
}
```

```
// Función flecha (arrow function) - MÁS COMÚN EN BACKEND
const sumar = (a, b) => {
  return a + b
}
```

```
// Función flecha simplificada (return implícito)
const sumar = (a, b) => a + b
```

```
// Función async (para operaciones asíncronas con base de datos)
const obtenerUsuario = async (id) => {
  const usuario = await User.findByPk(id)
```

```
return usuario
}
```

2.3. Promesas y Async/Await

Las operaciones con base de datos son **asíncronas** (no bloquean el código).

```
// MAL: Sin await (devuelve una Promise, no el resultado)
const restaurante = Restaurant.findByPk(1)
console.log(restaurante) // Promise { <pending> }
```

```
// BIEN: Con await (espera el resultado)
const restaurante = await Restaurant.findByPk(1)
console.log(restaurante) // { id: 1, name: 'Pizzería', ... }
```

```
// Siempre usa async/await en funciones que hacen consultas
const miControlador = async (req, res) => {
  try {
    const datos = await Model.findAll()
    res.json(datos)
  } catch (error) {
    res.status(500).send(error)
  }
}
```

2.4. Destructuring (Desestructuración)

```
// Extraer propiedades de un objeto
const usuario = { nombre: 'Ana', edad: 25, ciudad: 'Sevilla' }
```

```
// Sin destructuring
const nombre = usuario.nombre
const edad = usuario.edad
```

```
// Con destructuring (MÁS COMÚN)
const { nombre, edad } = usuario
```

```
// En parámetros de función
const mostrarUsuario = ({ nombre, edad }) => {
  console.log(`${nombre} tiene ${edad} años`)
}
```

2.5. Template Literals (Plantillas de cadenas)

```
// Concatenación antigua
const mensaje = 'Hola ' + nombre + ', tienes ' + edad + ' años'

// Template literal (MÁS COMÚN)
const mensaje = `Hola ${nombre}, tienes ${edad} años`

// Permite múltiples líneas
const html = `
```

`${titulo}`

`${contenido}`

2.6. Métodos de Arrays

```
const numeros = [1, 2, 3, 4, 5]

// map: transforma cada elemento
const dobles = numeros.map(n => n * 2) // [2, 4, 6, 8, 10]

// filter: filtra elementos
const pares = numeros.filter(n => n % 2 === 0) // [2, 4]

// find: encuentra el primer elemento que cumple la condición
const primero = numeros.find(n => n > 3) // 4

// some: verifica si al menos uno cumple la condición
const hayPares = numeros.some(n => n % 2 === 0) // true

// every: verifica si todos cumplen la condición
const todosPares = numeros.every(n => n % 2 === 0) // false
```

2.7. Operador Spread (...)

```
// Copiar arrays u objetos
const original = [1, 2, 3]
const copia = [...original] // [1, 2, 3]
```

```
const obj1 = { a: 1, b: 2 }
const obj2 = { ...obj1, c: 3 } // { a: 1, b: 2, c: 3 }

// Combinar arrays
const array1 = [1, 2]
const array2 = [3, 4]
const combinado = [...array1, ...array2] // [1, 2, 3, 4]
```

2.8. Manejo de Errores

```
// try-catch para capturar errores
const miFuncion = async (req, res) => {
  try {
    // Código que puede fallar
    const resultado = await operacionPeligrosa()
    res.json(resultado)
  } catch (error) {
    // Si hay error, lo capturamos aquí
    res.status(500).send(error)
  }
}
```

2.9. Exports/Imports (Módulos)

```
// Exportar desde un archivo (controllers/UserController.js)
const index = async (req, res) => { /* ... */ }
const create = async (req, res) => { /* ... */ }

// Exportar múltiples funciones
exports.index = index
exports.create = create

// O con destructuring
module.exports = {
  index,
  create
}

// Importar en otro archivo (routes/UserRoutes.js)
const UserController = require('../controllers/UserController')
```

```
// Usar las funciones
UserController.index
UserController.create
```

3. Arquitectura del Proyecto Backend

Estructura de Carpetas

DeliverUS-Backend/

```
|— src/
|   |— controllers/ # Lógica de negocio
|   |   |— RestaurantController.js
|   |   |— ReviewController.js
|   |   |— validation/ # Validaciones
|   |   |   |— RestaurantValidation.js
|   |   |   |— ReviewValidation.js
|   |— database/
|   |   |— migrations/ # Cambios en estructura BD
|   |   |   |— YYYYMMDDHHMMSS-create-review.js
|   |   |— seeders/ # Datos de prueba
|   |— middlewares/ # Funciones intermedias
|   |   |— AuthMiddleware.js
|   |   |— ReviewMiddleware.js
|   |— models/ # Definición de entidades
|   |   |— User.js
|   |   |— Restaurant.js
|   |   |— Review.js
|   |— routes/ # Definición de endpoints
|   |   |— RestaurantRoutes.js
|   |   |— ReviewRoutes.js
|— package.json
|— .env # Variables de entorno
```

Flujo de una Petición HTTP

1. Cliente hace petición → GET /restaurants/1/reviews
2. Router (routes/ReviewRoutes.js)
 - Identifica la ruta y los middlewares
 - Aplica middlewares de autenticación, validación, etc.

3. Validación (validation/ReviewValidation.js)
 - Valida los datos de entrada
 - Si hay errores, devuelve 422
 4. Middleware (middlewares/ReviewMiddleware.js)
 - Comprobaciones de seguridad/consistencia
 - Ej: ¿El usuario es dueño del recurso?
 5. Controller (controllers/ReviewController.js)
 - Ejecuta la lógica de negocio
 - Consulta la base de datos (Model)
 - Devuelve respuesta al cliente
 6. Respuesta → JSON con los datos o código de error
-

4. Migraciones de Base de Datos (Sequelize)

Las **migraciones** definen los cambios en la estructura de la base de datos (tablas, columnas, relaciones).

4.1. Estructura de una Migración

```
'use strict'
```

```
module.exports = {  
  // Función para aplicar cambios (crear tabla, añadir columna, etc.)  
  up: async (queryInterface, Sequelize) => {  
    await queryInterface.createTable('Reviews', {  
      // Columnas de la tabla  
    })  
  },  
  
  // Función para deshacer cambios (eliminar tabla, quitar columna,  
  etc.)  
  down: async (queryInterface, Sequelize) => {  
    await queryInterface.dropTable('Reviews')  
  }  
}
```

4.2. Crear una Nueva Tabla

'use strict'

```
module.exports = {  
  up: async (queryInterface, Sequelize) => {  
    await queryInterface.createTable('Reviews', {  
      // ID autoincremental (clave primaria)  
      id: {  
        allowNull: false,  
        autoIncrement: true,  
        primaryKey: true,  
        type: Sequelize.INTEGER  
      },
```

```
      // Columnas de datos  
      stars: {  
        type: Sequelize.INTEGER,  
        allowNull: false // No puede ser null (obligatorio)  
      },  
  
      body: {  
        type: Sequelize.TEXT,  
        allowNull: true // Puede ser null (opcional)  
      },  
  
      // Clave foránea hacia Restaurant  
      restaurantId: {  
        type: Sequelize.INTEGER,  
        allowNull: false,  
        references: {  
          model: 'Restaurants', // Nombre de la tabla referenciada  
          key: 'id'           // Columna referenciada  
        },  
        onDelete: 'CASCADE' // Si se borra el restaurante, se borran sus reviews  
      },  
  
      // Clave foránea hacia User (Customer)
```



```

customerId: {
  type: Sequelize.INTEGER,
  allowNull: false,
  references: {
    model: 'Users',
    key: 'id'
  },
  onDelete: 'CASCADE'
},

// Columnas de auditoría (siempre incluir)
createdAt: {
  allowNull: false,
  type: Sequelize.DATE,
  defaultValue: new Date()
},

updatedAt: {
  allowNull: false,
  type: Sequelize.DATE,
  defaultValue: new Date()
}
})

```

```

},

```

```

down: async (queryInterface, Sequelize) => {
  await queryInterface.dropTable('Reviews')
}
}

```

4.3. Tipos de Datos Comunes

// Enteros

Sequelize.INTEGER // Número entero

Sequelize.DOUBLE // Número decimal

// Texto

Sequelize.STRING // Texto corto (VARCHAR 255)

```
Sequelize.STRING(100) // Texto limitado a 100 caracteres
Sequelize.TEXT // Texto largo
```

```
// Booleanos
Sequelize.BOOLEAN // true/false
```

```
// Fechas y horas
Sequelize.DATE // Fecha y hora completa
Sequelize.TIME // Solo hora (HH:mm:ss)
```

```
// Enumerados
Sequelize.ENUM('online', 'offline', 'closed') // Solo estos valores
```

4.4. Añadir Columna a Tabla Existente

```
'use strict'

module.exports = {
  up: async (queryInterface, Sequelize) => {
    await queryInterface.addColumn(
      'Products', // Nombre de la tabla
      'scheduleId', // Nombre de la nueva columna
      {
        type: Sequelize.INTEGER,
        allowNull: true,
        references: {
          model: 'Schedules',
          key: 'id'
        },
        onDelete: 'SET NULL' // Si se borra el schedule, pone null
      }
    )
  },
  down: async (queryInterface, Sequelize) => {
    await queryInterface.removeColumn('Products', 'scheduleId')
  }
}
```

4.5. Añadir Columna Calculada (Virtual)

Para columnas calculadas como avgStars, NO se añaden en migraciones (se calculan en el modelo).

5. Modelos (Models)

Los **modelos** definen la estructura de datos y las relaciones entre entidades.

5.1. Estructura Básica de un Modelo

```
'use strict'
const { Model } = require('sequelize')

module.exports = (sequelize, DataTypes) => {
  class Review extends Model {
    static associate (models) {
      // Definir relaciones aquí
      Review.belongsTo(models.Restaurant, { foreignKey: 'restaurantId', as:
      'restaurant' })
      Review.belongsTo(models.User, { foreignKey: 'customerId', as:
      'customer' })
    }
  }

  Review.init({
    // Definición de columnas
    id: {
      type: DataTypes.INTEGER,
      autoIncrement: true,
      primaryKey: true
    },
    stars: {
      type: DataTypes.INTEGER,
      allowNull: false,
      validate: {
        min: 0,
        max: 5
      }
    }
  })
}
```

```

},
body: {
  type: DataTypes.TEXT,
  allowNull: true
},
restaurantId: {
  type: DataTypes.INTEGER,
  allowNull: false
},
customerId: {
  type: DataTypes.INTEGER,
  allowNull: false
}
}, {
  sequelize,
  modelName: 'Review',
  tableName: 'Reviews'
})

return Review
}

```

5.2. Tipos de Relaciones

5.2.1. belongsTo (Pertenece a / N:1)

Una review **pertenece a** un restaurante.

```

// En Review.js
Review.belongsTo(models.Restaurant, {
  foreignKey: 'restaurantId',
  as: 'restaurant'
})

```

```

// Esto permite:
const review = await Review.findByPk(1, {
  include: { model: Restaurant, as: 'restaurant' }
})
console.log(review.restaurant.name) // Acceder al restaurante de la
review

```

5.2.2. hasMany (Tiene muchos / 1:N)

Un restaurante **tiene muchas** reviews.

```
// En Restaurant.js
Restaurant.hasMany(models.Review, {
  foreignKey: 'restaurantId',
  as: 'reviews'
})

// Esto permite:
const restaurant = await Restaurant.findByPk(1, {
  include: { model: Review, as: 'reviews' }
})
console.log(restaurant.reviews.length) // Número de reviews
```

5.2.3. belongsToMany (Muchos a muchos / N:M)

Relación a través de tabla intermedia (ejemplo: Order y Product).

```
// Order y Product se relacionan mediante OrderProducts
Order.belongsToMany(models.Product, {
  through: 'OrderProducts',
  foreignKey: 'orderId'
})

Product.belongsToMany(models.Order, {
  through: 'OrderProducts',
  foreignKey: 'productId'
})
```

5.3. Validaciones en el Modelo

```
Review.init({
  stars: {
    type: DataTypes.INTEGER,
    allowNull: false,
    validate: {
      // Validación personalizada
      min: {
        args: [0],
        msg: 'Stars must be at least 0'
      }
    }
  }
})
```

```
},  
max: {  
  args: [5],  
  msg: 'Stars must be at most 5'  
},  
isInt: {  
  msg: 'Stars must be an integer'  
}  
}  
},
```

```
email: {  
  type: DataTypes.STRING,  
  validate: {  
    isEmail: {  
      msg: 'Must be a valid email'  
    }  
  }  
},
```

```
url: {  
  type: DataTypes.STRING,  
  validate: {  
    isUrl: {  
      msg: 'Must be a valid URL'  
    }  
  }  
}  
})
```

5.4. Atributos Virtuales (Calculados)

Columnas que NO existen en la base de datos, pero se calculan dinámicamente.

```
// En Restaurant.js  
Restaurant.init({  
  // ... otras columnas ...
```

```
  // Atributo virtual  
  avgStars: {
```

```

type: DataTypes.VIRTUAL,
get () {
return this.getAvgStars()
}
}, {
sequelize,
modelName: 'Restaurant'
})

// Función para calcular el promedio
Restaurant.prototype.getAvgStars = function () {
// Si no hay reviews, devolver null
if (!this.reviews || this.reviews.length === 0) {
return null
}

// Calcular promedio
const sum = this.reviews.reduce((acc, review) => acc + review.stars, 0)
const avg = sum / this.reviews.length
return parseFloat(avg.toFixed(2)) // Redondear a 2 decimales
}

```

6. Controladores (Controllers)

Los **controladores** contienen la lógica de negocio de cada endpoint.

6.1. Estructura de un Controlador

```

'use strict'
const { Review, Restaurant, User } = require('../models')

// Listar todas las reviews de un restaurante
const index = async (req, res) => {
try {
const reviews = await Review.findAll({
where: { restaurantId: req.params.restaurantId }
})
res.json(reviews)
} catch (err) {

```

```
res.status(500).send(err)
}
}
```

```
// Crear una nueva review
const create = async (req, res) => {
  try {
    const newReview = await Review.create({
      ...req.body,
      customerId: req.user.id,
      restaurantId: req.params.restaurantId
    })
    res.json(newReview)
  } catch (err) {
    res.status(500).send(err)
  }
}
```

```
// Actualizar una review
const update = async (req, res) => {
  try {
    await Review.update(req.body, {
      where: { id: req.params.reviewId }
    })
    const updatedReview = await Review.findByPk(req.params.reviewId)
    res.json(updatedReview)
  } catch (err) {
    res.status(500).send(err)
  }
}
```

```
// Eliminar una review
const destroy = async (req, res) => {
  try {
    await Review.destroy({
      where: { id: req.params.reviewId }
    })
    res.json({ message: 'Review deleted successfully' })
  } catch (err) {
    res.status(500).send(err)
  }
}
```



```
}  
}  
  
module.exports = {  
  index,  
  create,  
  update,  
  destroy  
}
```

6.2. Consultas con Sequelize

6.2.1. findAll (Obtener todos)

```
// Obtener todos los registros  
const reviews = await Review.findAll()  
  
// Con condición WHERE  
const reviews = await Review.findAll({  
  where: { restaurantId: 1 }  
})  
  
// Con múltiples condiciones  
const reviews = await Review.findAll({  
  where: {  
    restaurantId: 1,  
    stars: 5  
  }  
})  
  
// Ordenar resultados  
const reviews = await Review.findAll({  
  order: [['createdAt', 'DESC']] // Más recientes primero  
})
```

6.2.2. findByPk (Obtener por clave primaria)

```
// Buscar por ID  
const review = await Review.findByPk(1)  
  
// Con relaciones incluidas  
const review = await Review.findByPk(1, {
```

```
include: [  
  { model: Restaurant, as: 'restaurant' },  
  { model: User, as: 'customer' }  
]  
})
```

6.2.3. findOne (Obtener uno)

```
// Buscar el primero que cumple la condición  
const review = await Review.findOne({  
  where: {  
    restaurantId: 1,  
    customerId: 5  
  }  
})
```

6.2.4. create (Crear)

```
// Crear nuevo registro  
const newReview = await Review.create({  
  stars: 5,  
  body: 'Excelente comida',  
  restaurantId: 1,  
  customerId: req.user.id  
})
```

```
// Con spread operator (datos del body)  
const newReview = await Review.create({  
  ...req.body,  
  customerId: req.user.id,  
  restaurantId: req.params.restaurantId  
})
```

6.2.5. update (Actualizar)

```
// Actualizar registro  
await Review.update(  
  { stars: 4, body: 'Buena comida' }, // Datos a actualizar  
  { where: { id: 1 } } // Condición  
)
```

```
// Con datos del body
await Review.update(req.body, {
  where: { id: req.params.reviewId }
})
```

6.2.6. destroy (Eliminar)

```
// Eliminar registro
await Review.destroy({
  where: { id: 1 }
})
```

```
// Eliminar múltiples
await Review.destroy({
  where: { restaurantId: 1 }
})
```

6.3. Incluir Relaciones (include)

```
// Incluir una relación
const restaurant = await Restaurant.findByPk(1, {
  include: { model: Review, as: 'reviews' }
})
```

```
// Incluir múltiples relaciones
const restaurant = await Restaurant.findByPk(1, {
  include: [
    { model: Review, as: 'reviews' },
    { model: Product, as: 'products' },
    { model: User, as: 'owner' }
  ]
})
```

```
// Incluir con condiciones
const restaurant = await Restaurant.findByPk(1, {
  include: {
    model: Review,
    as: 'reviews',
    where: { stars: 5 }
  }
})
```

6.4. Funciones Útiles de Sequelize

```
// Contar registros
const count = await Review.count({
  where: { restaurantId: 1 }
})

// Sumar valores
const sum = await Review.sum('stars', {
  where: { restaurantId: 1 }
})

// Máximo y mínimo
const max = await Review.max('stars')
const min = await Review.min('stars')

// Comprobar si existe
const exists = await Review.findOne({
  where: { customerId: 1, restaurantId: 1 }
})
if (exists) {
  // Ya existe
}
```

7. Rutas (Routes)

Las **rutas** definen los endpoints de la API y los middlewares que se aplican.

7.1. Estructura de un Archivo de Rutas

```
'use strict'
const ReviewController = require('../controllers/ReviewController')
const { isLoggedIn, hasRole } =
  require('../middlewares/AuthMiddleware')
const { checkReviewOwnership } =
  require('../middlewares/ReviewMiddleware')
const { checkEntityExists } =
  require('../middlewares/EntityMiddleware')
const { handleValidation } =
```

```
require('../middlewares/ValidationHandlingMiddleware')
const { create, update } =
require('../controllers/validation/ReviewValidation')
const { Review, Restaurant } = require('../models')

module.exports = (options) => {
  const app = options.app
  const middlewares = options.middlewares

  // GET /restaurants/:restaurantId/reviews - Listar reviews
  app.route('/restaurants/:restaurantId/reviews')
    .get(
      ReviewController.index
    )
    .post(
      isLoggedIn,
      hasRole('customer'),
      checkEntityExists(Restaurant, 'restaurantId'),
      create,
      handleValidation,
      ReviewController.create
    )

  // PUT /restaurants/:restaurantId/reviews/:reviewId - Actualizar
  review
  // DELETE /restaurants/:restaurantId/reviews/:reviewId - Eliminar
  review
  app.route('/restaurants/:restaurantId/reviews/:reviewId')
    .put(
      isLoggedIn,
      hasRole('customer'),
      checkEntityExists(Restaurant, 'restaurantId'),
      checkEntityExists(Review, 'reviewId'),
      checkReviewOwnership,
      update,
      handleValidation,
      ReviewController.update
    )
    .delete(
      isLoggedIn,
```

```
hasRole('customer'),
checkEntityExists(Restaurant, 'restaurantId'),
checkEntityExists(Review, 'reviewId'),
checkReviewOwnership,
ReviewController.destroy
)
}
```

7.2. Tipos de Rutas HTTP

```
// GET - Obtener datos (listar, mostrar)
app.route('/reviews').get(ReviewController.index)
app.route('/reviews/:id').get(ReviewController.show)

// POST - Crear nuevo recurso
app.route('/reviews').post(ReviewController.create)

// PUT - Actualizar recurso completo
app.route('/reviews/:id').put(ReviewController.update)

// PATCH - Actualizar parcialmente
app.route('/reviews/:id/default').patch(ReviewController.setDefault)

// DELETE - Eliminar recurso
app.route('/reviews/:id').delete(ReviewController.destroy)
```

7.3. Parámetros en Rutas

```
// Parámetros de ruta (params)
app.route('/restaurants/:restaurantId/reviews/:reviewId')
// Acceso: req.params.restaurantId, req.params.reviewId

// Query strings (query)
// URL: /reviews?stars=5&limit=10
// Acceso: req.query.stars, req.query.limit

// Body (datos enviados en POST/PUT)
// Acceso: req.body.stars, req.body.body
```

7.4. Orden de Middlewares (IMPORTANTE)

El orden de los middlewares importa. Se ejecutan de arriba a abajo:

```
app.route('/restaurants/:restaurantId/reviews')
.post(
// 1. Verificar autenticación
isLoggedIn,
```

```
// 2. Verificar rol
hasRole('customer'),

// 3. Verificar que existe el restaurante
checkEntityExists(Restaurant, 'restaurantId'),

// 4. Validar datos de entrada
create,
handleValidation,

// 5. Verificaciones de negocio
userHasPlacedOrderInRestaurant,
checkCustomerHasNotReviewed,

// 6. Finalmente, ejecutar el controlador
ReviewController.create
```

```
)
```

8. Middlewares

Los **middlewares** son funciones que se ejecutan antes del controlador para hacer comprobaciones.

8.1. Estructura de un Middleware

```
const miMiddleware = async (req, res, next) => {
try {
// Hacer comprobaciones
if (condicionFalla) {
```

```
return res.status(403).send('Forbidden')
}
```

```
// Si todo está bien, continuar al siguiente middleware
next()
```

```
} catch (err) {
return res.status(500).send(err)
}
}
```

```
module.exports = {
miMiddleware
}
```

8.2. Middlewares de Autenticación

```
// isLoggedIn: Verifica que el usuario está autenticado
const isLoggedIn = async (req, res, next) => {
if (!req.user) {
return res.status(401).send('Unauthorized')
}
next()
}
```

```
// hasRole: Verifica que el usuario tiene un rol específico
const hasRole = (role) => {
return async (req, res, next) => {
if (req.user.userType !== role) {
return res.status(403).send('Forbidden')
}
next()
}
}
```

```
// Uso:
// isLoggedIn
// hasRole('customer')
// hasRole('owner')
```


8.3. Middleware para Verificar Existencia

```
// Verifica que una entidad existe
const checkEntityExists = (model, paramName) => {
  return async (req, res, next) => {
    try {
      const entity = await model.findByPk(req.params[paramName])
      if (!entity) {
        return res.status(404).send(`${model.name} not found`)
      }
      next()
    } catch (err) {
      return res.status(500).send(err)
    }
  }
}

// Uso:
// checkEntityExists(Restaurant, 'restaurantId')
// checkEntityExists(Review, 'reviewId')
```

8.4. Middleware de Propiedad (Ownership)

```
// Verifica que el usuario es dueño del recurso
const checkReviewOwnership = async (req, res, next) => {
  try {
    const review = await Review.findByPk(req.params.reviewId)
```

```
    if (!review) {
      return res.status(404).send('Review not found')
    }

    if (review.customerId !== req.user.id) {
      return res.status(403).send('You are not the owner of this review')
    }

    next()
  }
}
```

```
} catch (err) {  
  return res.status(500).send(err)  
}  
}
```

8.5. Middleware de Lógica de Negocio

```
// Verifica que el cliente ha hecho pedidos en el restaurante  
const userHasPlacedOrderInRestaurant = async (req, res, next) => {  
  try {  
    const order = await Order.findOne({  
      where: {  
        customerId: req.user.id,  
        restaurantId: req.params.restaurantId  
      }  
    })
```

```
    if (!order) {  
      return res.status(409).send('You must place an order before reviewing')  
    }  
  
    next()
```

```
  } catch (err) {  
    return res.status(500).send(err)  
  }  
}
```

```
// Verifica que el cliente no ha dejado ya una review  
const checkCustomerHasNotReviewed = async (req, res, next) => {  
  try {  
    const existingReview = await Review.findOne({  
      where: {  
        customerId: req.user.id,  
        restaurantId: req.params.restaurantId  
      }  
    })
```

```

    if (existingReview) {
      return res.status(409).send('You have already reviewed this restaurant')
    }

    next()

  } catch (err) {
    return res.status(500).send(err)
  }
}

```

8.6. Middleware para Verificar Pertenencia (belongsTo)

```

// Verifica que el schedule pertenece al restaurante del producto
const checkScheduleBelongsToRestaurant = async (req, res, next) => {
  try {
    // Si no se especifica scheduleId, continuar (es opcional)
    if (!req.body.scheduleId) {
      return next()
    }
  }

```

```

    // Obtener el restaurantId
    let restaurantId
    if (req.params.restaurantId) {
      restaurantId = req.params.restaurantId
    } else if (req.body.restaurantId) {
      restaurantId = req.body.restaurantId
    } else {
      // Si estamos editando, obtener del producto existente
      const product = await Product.findByPk(req.params.productId)
      restaurantId = product.restaurantId
    }

    // Verificar que el schedule pertenece al restaurante
    const schedule = await Schedule.findOne({
      where: {
        id: req.body.scheduleId,

```

```
    restaurantId: restaurantId
  }
})

if (!schedule) {
  return res.status(422).send('Schedule does not belong to restaurant')
}

next()

} catch (err) {
return res.status(500).send(err)
}
}
```

9. Validaciones (express-validator)

Las **validaciones** verifican que los datos de entrada son correctos antes de procesarlos.

9.1. Estructura de Validaciones

```
'use strict'
const { check } = require('express-validator')

// Validaciones para crear
const create = [
  check('stars')
    .exists().withMessage('Stars is required')
    .isInt({ min: 0, max: 5 }).withMessage('Stars must be between 0 and 5'),

  check('body')
    .optional() // No es obligatorio
    .isString().withMessage('Body must be a string')
    .trim()
]

// Validaciones para actualizar
const update = [
```

```
check('stars')
.exists().withMessage('Stars is required')
.isInt({ min: 0, max: 5 }).withMessage('Stars must be between 0 and
5'),

check('body')
.optional()
.isString().withMessage('Body must be a string')
.trim()
]

module.exports = {
create,
update
}
```

9.2. Validadores Comunes

```
// exists: Campo debe existir
check('name').exists().withMessage('Name is required')

// isString: Debe ser texto
check('name').isString().withMessage('Name must be a string')

// isInt: Debe ser entero
check('stars').isInt().withMessage('Stars must be an integer')

// isInt con rango
check('stars').isInt({ min: 0, max: 5 }).withMessage('Stars must be 0-5')

// isFloat: Debe ser decimal
check('price').isFloat({ min: 0 }).withMessage('Price must be positive')

// isEmail: Debe ser email válido
check('email').isEmail().withMessage('Must be a valid email')

// isURL: Debe ser URL válida
check('url').isURL().withMessage('Must be a valid URL')

// isBoolean: Debe ser true/false
check('isDefault').isBoolean().withMessage('Must be boolean')
```

```

// isIn: Debe estar en lista de valores
check('status').isIn(['online', 'offline', 'closed'])
.withMessage('Status must be online, offline or closed')

// optional: Campo no obligatorio
check('description').optional().isString()

// trim: Eliminar espacios al inicio/final
check('name').trim()

// custom: Validación personalizada
check('endTime').custom((value, { req }) => {
  if (value <= req.body.startTime) {
    throw new Error('End time must be after start time')
  }
  return true
})

```

9.3. Validaciones Personalizadas

```

// Validar formato de hora (HH:mm:ss)
const validateTimeFormat = (value) => {
  const timeRegex = /^[01]\d|2[0-3]:([0-5]\d):([0-5]\d)$/
  if (!timeRegex.test(value)) {
    throw new Error('Time must be in HH:mm:ss format')
  }
  return true
}

// Validar que endTime es posterior a startTime
const validateEndTimeAfterStartTime = (endTime, { req }) => {
  if (endTime <= req.body.startTime) {
    throw new Error('End time must be after start time')
  }
  return true
}

// Uso en validaciones
const create = [
  check('startTime')

```

```
.exists().withMessage('Start time is required')
.custom(validateTimeFormat),

check('endTime')
.exists().withMessage('End time is required')
.custom(validateTimeFormat)
.custom(validateEndTimeAfterStartTime)
]
```

9.4. Validación Asíncrona (Base de Datos)

```
// Verificar que el schedule pertenece al restaurante
check('scheduleId')
.optional()
.custom(async (value, { req }) => {
  if (!value) return true
```

```
const schedule = await Schedule.findOne({
  where: {
    id: value,
    restaurantId: req.body.restaurantId || req.params.restaurantId
  }
})

if (!schedule) {
  throw new Error('Schedule does not belong to restaurant')
}
return true
```

```
})
```

10. Ejemplos Completos Resueltos

10.1. Ejemplo 1: Reviews (Examen Marzo 2025)

10.1.1. Migración - create-review.js

```
'use strict'

module.exports = {
  up: async (queryInterface, Sequelize) => {
    await queryInterface.createTable('Reviews', {
      id: {
        allowNull: false,
        autoIncrement: true,
        primaryKey: true,
        type: Sequelize.INTEGER
      },
      stars: {
        type: Sequelize.INTEGER,
        allowNull: false
      },
      body: {
        type: Sequelize.TEXT,
        allowNull: true
      },
      restaurantId: {
        type: Sequelize.INTEGER,
        allowNull: false,
        references: {
          model: 'Restaurants',
          key: 'id'
        },
        onDelete: 'CASCADE'
      },
      customerId: {
        type: Sequelize.INTEGER,
        allowNull: false,
        references: {
          model: 'Users',
          key: 'id'
        },
        onDelete: 'CASCADE'
      },
      createdAt: {
```



```

allowNull: false,
type: Sequelize.DATE,
defaultValue: new Date()
},
updatedAt: {
allowNull: false,
type: Sequelize.DATE,
defaultValue: new Date()
}
})
},

down: async (queryInterface, Sequelize) => {
await queryInterface.dropTable('Reviews')
}
}

```

10.1.2. Modelo - Review.js

```

'use strict'
const { Model } = require('sequelize')

module.exports = (sequelize, DataTypes) => {
class Review extends Model {
static associate (models) {
// Review pertenece a Restaurant
Review.belongsTo(models.Restaurant, {
foreignKey: 'restaurantId',
as: 'restaurant'
})
}
}

// Review pertenece a User (Customer)
Review.belongsTo(models.User, {
foreignKey: 'customerId',
as: 'customer'
})
}
}

```

```

Review.init({
  id: {
    type: DataTypes.INTEGER,
    autoIncrement: true,
    primaryKey: true
  },
  stars: {
    type: DataTypes.INTEGER,
    allowNull: false,
    validate: {
      min: 0,
      max: 5
    }
  },
  body: {
    type: DataTypes.TEXT,
    allowNull: true
  },
  restaurantId: {
    type: DataTypes.INTEGER,
    allowNull: false
  },
  customerId: {
    type: DataTypes.INTEGER,
    allowNull: false
  }
}, {
  sequelize,
  modelName: 'Review',
  tableName: 'Reviews'
})

return Review
}

```

10.1.3. Modelo - Restaurant.js (Modificaciones)

```

'use strict'
const { Model } = require('sequelize')

```

```
module.exports = (sequelize, DataTypes) => {  
  class Restaurant extends Model {  
    static associate (models) {  
      // ... asociaciones existentes ...
```

```
      // Restaurant tiene muchas Reviews  
      Restaurant.hasMany(models.Review, {  
        foreignKey: 'restaurantId',  
        as: 'reviews'  
      })  
    }  
  
    // Función para calcular avgStars  
    getAvgStars () {  
      if (!this.reviews || this.reviews.length === 0) {  
        return null  
      }  
  
      const sum = this.reviews.reduce((acc, review) => acc + review.stars, 0)  
      const avg = sum / this.reviews.length  
      return parseFloat(avg.toFixed(2))  
    }  
  }  
}
```

```
  }  
  
  Restaurant.init({  
    // ... campos existentes ...
```

```
    // Atributo virtual avgStars  
    avgStars: {  
      type: DataTypes.VIRTUAL,  
      get () {  
        return this.getAvgStars()  
      }  
    }  
  }  
}
```

```

}, {
  sequelize,
  modelName: 'Restaurant',
  tableName: 'Restaurants'
})

return Restaurant
}

```

10.1.4. Rutas - ReviewRoutes.js

```

'use strict'
const ReviewController = require('../controllers/ReviewController')
const { isLoggedIn, hasRole } =
  require('../middlewares/AuthMiddleware')
const { checkReviewOwnership, userHasPlacedOrderInRestaurant,
  checkCustomerHasNotReviewed } =
  require('../middlewares/ReviewMiddleware')
const { checkEntityExists } =
  require('../middlewares/EntityMiddleware')
const { handleValidation } =
  require('../middlewares/ValidationHandlingMiddleware')
const { create, update } =
  require('../controllers/validation/ReviewValidation')
const { Review, Restaurant } = require('../models')

module.exports = (options) => {
  const app = options.app

  // GET /restaurants/:restaurantId/reviews - Listar reviews
  app.route('/restaurants/:restaurantId/reviews')
    .get(
      checkEntityExists(Restaurant, 'restaurantId'),
      ReviewController.index
    )
    .post(
      isLoggedIn,
      hasRole('customer'),
      checkEntityExists(Restaurant, 'restaurantId'),
      create,
      handleValidation,

```

```

userHasPlacedOrderInRestaurant,
checkCustomerHasNotReviewed,
ReviewController.create
)

// PUT/DELETE /restaurants/:restaurantId/reviews/:reviewId
app.route('/restaurants/:restaurantId/reviews/:reviewId')
.put(
  isLoggedIn,
  hasRole('customer'),
  checkEntityExists(Restaurant, 'restaurantId'),
  checkEntityExists(Review, 'reviewId'),
  update,
  handleValidation,
  checkReviewOwnership,
  ReviewController.update
)
.delete(
  isLoggedIn,
  hasRole('customer'),
  checkEntityExists(Restaurant, 'restaurantId'),
  checkEntityExists(Review, 'reviewId'),
  checkReviewOwnership,
  ReviewController.destroy
)
}

```

10.1.5. Validaciones - ReviewValidation.js

```

'use strict'
const { check } = require('express-validator')

const create = [
  check('stars')
    .exists().withMessage('Stars is required')
    .isInt({ min: 0, max: 5 }).withMessage('Stars must be between 0 and 5'),

  check('body')
    .optional()
    .isString().withMessage('Body must be a string')

```

```

.trim()
]

const update = [
  check('stars')
  .exists().withMessage('Stars is required')
  .isInt({ min: 0, max: 5 }).withMessage('Stars must be between 0 and
  5'),

  check('body')
  .optional()
  .isString().withMessage('Body must be a string')
  .trim()
]

module.exports = {
  create,
  update
}

```

10.1.6. Controlador - ReviewController.js

```

'use strict'
const { Review, Restaurant, User } = require('../models')

// Listar todas las reviews de un restaurante
const index = async (req, res) => {
  try {
    const reviews = await Review.findAll({
      where: { restaurantId: req.params.restaurantId },
      order: [['createdAt', 'DESC']]
    })
    res.json(reviews)
  } catch (err) {
    res.status(500).send(err)
  }
}

// Crear una nueva review
const create = async (req, res) => {
  try {

```

```

const newReview = await Review.create({
  stars: req.body.stars,
  body: req.body.body,
  customerId: req.user.id,
  restaurantId: req.params.restaurantId
})
res.json(newReview)
} catch (err) {
  res.status(500).send(err)
}
}

// Actualizar una review
const update = async (req, res) => {
  try {
    await Review.update(
      {
        stars: req.body.stars,
        body: req.body.body
      },
      {
        where: { id: req.params.reviewId }
      }
    )
    const updatedReview = await Review.findByPk(req.params.reviewId)
    res.json(updatedReview)
  } catch (err) {
    res.status(500).send(err)
  }
}

// Eliminar una review
const destroy = async (req, res) => {
  try {
    const result = await Review.destroy({
      where: { id: req.params.reviewId }
    })
    res.json(result)
  } catch (err) {
    res.status(500).send(err)
  }
}

```

```
}  
}
```

```
module.exports = {  
  index,  
  create,  
  update,  
  destroy  
}
```

10.1.7. Middlewares - ReviewMiddleware.js

```
'use strict'  
const { Review, Order } = require('../models')  
  
// Verificar que el usuario es dueño de la review  
const checkReviewOwnership = async (req, res, next) => {  
  try {  
    const review = await Review.findById(req.params.reviewId)
```

```
    if (!review) {  
      return res.status(404).send('Review not found')  
    }  
  
    // Verificar que pertenece al restaurante  
    if (review.restaurantId !== parseInt(req.params.restaurantId)) {  
      return res.status(409).send('Review does not belong to this restaurant')  
    }  
  
    // Verificar que el usuario es el dueño  
    if (review.customerId !== req.user.id) {  
      return res.status(403).send('You are not the owner of this review')  
    }  
  
    next()
```

```
  } catch (err) {  
    return res.status(500).send(err)
```



```
}  
}
```

```
// Verificar que el usuario ha hecho pedidos en el restaurante  
const userHasPlacedOrderInRestaurant = async (req, res, next) => {  
  try {  
    const order = await Order.findOne({  
      where: {  
        customerId: req.user.id,  
        restaurantId: req.params.restaurantId  
      }  
    })  
  }
```

```
    if (!order) {  
      return res.status(409).send('You must have placed an order to review this restaurant')  
    }  
  
    next()  
  }
```

```
  } catch (err) {  
    return res.status(500).send(err)  
  }  
}
```

```
// Verificar que el cliente no ha dejado ya una review  
const checkCustomerHasNotReviewed = async (req, res, next) => {  
  try {  
    const existingReview = await Review.findOne({  
      where: {  
        customerId: req.user.id,  
        restaurantId: req.params.restaurantId  
      }  
    })  
  }
```

```
    if (existingReview) {  
      return res.status(409).send('You have already reviewed this restaurant')  
    }  
  }
```

```
next()
```

```
} catch (err) {  
return res.status(500).send(err)  
}  
}
```

```
module.exports = {  
  checkReviewOwnership,  
  userHasPlacedOrderInRestaurant,  
  checkCustomerHasNotReviewed  
}
```

10.2. Ejemplo 2: ShippingAddress (Examen Julio 2025)

10.2.1. Migración - create-shippingaddress.js

```
'use strict'
```

```
module.exports = {  
  up: async (queryInterface, Sequelize) => {  
    await queryInterface.createTable('ShippingAddresses', {  
      id: {  
        allowNull: false,  
        autoIncrement: true,  
        primaryKey: true,  
        type: Sequelize.INTEGER  
      },  
      alias: {  
        type: Sequelize.STRING,  
        allowNull: false  
      },  
      street: {  
        type: Sequelize.STRING,  
        allowNull: false  
      },  
      city: {  
        type: Sequelize.STRING,  
        allowNull: false  
      }  
    })  
  }  
}
```

```

},
zipCode: {
  type: Sequelize.STRING,
  allowNull: false
},
province: {
  type: Sequelize.STRING,
  allowNull: false
},
isDefault: {
  type: Sequelize.BOOLEAN,
  allowNull: false,
  defaultValue: false
},
userId: {
  type: Sequelize.INTEGER,
  allowNull: false,
  references: {
    model: 'Users',
    key: 'id'
  },
  onDelete: 'CASCADE'
},
createdAt: {
  allowNull: false,
  type: Sequelize.DATE,
  defaultValue: new Date()
},
updatedAt: {
  allowNull: false,
  type: Sequelize.DATE,
  defaultValue: new Date()
}
})
},

down: async (queryInterface, Sequelize) => {
  await queryInterface.dropTable('ShippingAddresses')
}
}

```

10.2.2. Modelo - ShippingAddress.js

```
'use strict'
const { Model } = require('sequelize')

module.exports = (sequelize, DataTypes) => {
  class ShippingAddress extends Model {
    static associate (models) {
      ShippingAddress.belongsTo(models.User, {
        foreignKey: 'userId',
        as: 'user'
      })
    }
  }
}

ShippingAddress.init({
  id: {
    type: DataTypes.INTEGER,
    autoIncrement: true,
    primaryKey: true
  },
  alias: {
    type: DataTypes.STRING,
    allowNull: false
  },
  street: {
    type: DataTypes.STRING,
    allowNull: false
  },
  city: {
    type: DataTypes.STRING,
    allowNull: false
  },
  zipCode: {
    type: DataTypes.STRING,
    allowNull: false
  },
  province: {
    type: DataTypes.STRING,
    allowNull: false
  }
})
```

```

},
isDefault: {
  type: DataTypes.BOOLEAN,
  allowNull: false,
  defaultValue: false
},
userId: {
  type: DataTypes.INTEGER,
  allowNull: false
}
}, {
  sequelize,
  modelName: 'ShippingAddress',
  tableName: 'ShippingAddresses'
})

return ShippingAddress
}

```

10.2.3. Rutas - ShippingAddressRoutes.js

```

'use strict'
const ShippingAddressController =
  require('../controllers/ShippingAddressController')
const { isLoggedIn, hasRole } =
  require('../middlewares/AuthMiddleware')
const { checkShippingAddressOwnership } =
  require('../middlewares/ShippingAddressMiddleware')
const { handleValidation } =
  require('../middlewares/ValidationHandlingMiddleware')
const { create } =
  require('../controllers/validation/ShippingAddressValidation')

module.exports = (options) => {
  const app = options.app

  // GET /shippingaddresses - Listar direcciones del usuario
  // POST /shippingaddresses - Crear nueva dirección
  app.route('/shippingaddresses')
    .get(
      isLoggedIn,

```

```

    hasRole('customer'),
    ShippingAddressController.index
  )
  .post(
    isLoggedIn,
    hasRole('customer'),
    create,
    handleValidation,
    ShippingAddressController.create
  )

  // PATCH /shippingaddresses/:shippingAddressId/default
  app.route('/shippingaddresses/:shippingAddressId/default')
  .patch(
    isLoggedIn,
    hasRole('customer'),
    checkShippingAddressOwnership,
    ShippingAddressController.setDefault
  )

  // DELETE /shippingaddresses/:shippingAddressId
  app.route('/shippingaddresses/:shippingAddressId')
  .delete(
    isLoggedIn,
    hasRole('customer'),
    checkShippingAddressOwnership,
    ShippingAddressController.destroy
  )
}

```

10.2.4. Validaciones - ShippingAddressValidation.js

```

'use strict'
const { check } = require('express-validator')

const create = [
  check('alias')
  .exists().withMessage('Alias is required')
  .isString().withMessage('Alias must be a string')
  .trim()
  .notEmpty().withMessage('Alias cannot be empty'),

```

```

check('street')
.exists().withMessage('Street is required')
.isString().withMessage('Street must be a string')
.trim()
.notEmpty().withMessage('Street cannot be empty'),

check('city')
.exists().withMessage('City is required')
.isString().withMessage('City must be a string')
.trim()
.notEmpty().withMessage('City cannot be empty'),

check('zipCode')
.exists().withMessage('Zip code is required')
.isString().withMessage('Zip code must be a string')
.trim()
.notEmpty().withMessage('Zip code cannot be empty'),

check('province')
.exists().withMessage('Province is required')
.isString().withMessage('Province must be a string')
.trim()
.notEmpty().withMessage('Province cannot be empty')
]

module.exports = {
  create
}

```

10.2.5. Controlador - ShippingAddressController.js

```

'use strict'
const { ShippingAddress } = require('../models')

// Listar direcciones del usuario autenticado
const index = async (req, res) => {
  try {
    const addresses = await ShippingAddress.findAll({
      where: { userId: req.user.id },
      order: [['isDefault', 'DESC'], ['createdAt', 'DESC']]
    })
  }
}

```

```
res.json(addresses)
} catch (err) {
res.status(500).send(err)
}
}
```

```
// Crear nueva dirección
const create = async (req, res) => {
try {
// Verificar si es la primera dirección del usuario
const existingAddresses = await ShippingAddress.count({
where: { userId: req.user.id }
})
```

```
// Si es la primera, marcarla como predeterminada
const isDefault = existingAddresses === 0

const newAddress = await ShippingAddress.create({
...req.body,
userId: req.user.id,
isDefault: isDefault
})
```

```
res.json(newAddress)
```

```
} catch (err) {
res.status(500).send(err)
}
}
```

```
// Marcar dirección como predeterminada
const setDefault = async (req, res) => {
try {
// Desmarcar todas las direcciones del usuario
await ShippingAddress.update(
{ isDefault: false },
{ where: { userId: req.user.id } }
)
```



```

// Marcar la dirección especificada como predeterminada
await ShippingAddress.update(
  { isDefault: true },
  { where: { id: req.params.shippingAddressId } }
)

const updatedAddress = await ShippingAddress.findByPk(req.params.shippingAddressId)
res.json(updatedAddress)

} catch (err) {
res.status(500).send(err)
}
}

// Eliminar dirección
const destroy = async (req, res) => {
try {
const result = await ShippingAddress.destroy({
where: { id: req.params.shippingAddressId }
})
res.json(result)
} catch (err) {
res.status(500).send(err)
}
}

module.exports = {
index,
create,
setDefault,
destroy
}

```

10.2.6. Middleware - ShippingAddressMiddleware.js

```

'use strict'
const { ShippingAddress } = require('../models')

// Verificar que la dirección pertenece al usuario
const checkShippingAddressOwnership = async (req, res, next) => {

```

```
try {  
  const address = await  
  ShippingAddress.findByPk(req.params.shippingAddressId)
```

```
  if (!address) {  
    return res.status(404).send('Shipping address not found')  
  }  
  
  if (address.userId !== req.user.id) {  
    return res.status(403).send('You are not the owner of this address')  
  }  
  
  next()
```

```
} catch (err) {  
  return res.status(500).send(err)  
}  
}
```

```
module.exports = {  
  checkShippingAddressOwnership  
}
```

10.3. Ejemplo 3: Schedules (Examen Marzo 2025 - Modelo B)

10.3.1. Migración - create-schedules.js

```
'use strict'
```

```
module.exports = {  
  up: async (queryInterface, Sequelize) => {  
    await queryInterface.createTable('Schedules', {  
      id: {  
        allowNull: false,  
        autoIncrement: true,  
        primaryKey: true,  
        type: Sequelize.INTEGER  
      },  

```

```

startTime: {
  type: Sequelize.TIME,
  allowNull: false
},
endTime: {
  type: Sequelize.TIME,
  allowNull: false
},
restaurantId: {
  type: Sequelize.INTEGER,
  allowNull: false,
  references: {
    model: 'Restaurants',
    key: 'id'
  },
  onDelete: 'CASCADE'
},
createdAt: {
  allowNull: false,
  type: Sequelize.DATE,
  defaultValue: new Date()
},
updatedAt: {
  allowNull: false,
  type: Sequelize.DATE,
  defaultValue: new Date()
}
})
},

down: async (queryInterface, Sequelize) => {
  await queryInterface.dropTable('Schedules')
}
}

```

10.3.2. Migración - Añadir scheduleId a Products

```

'use strict'

module.exports = {
  up: async (queryInterface, Sequelize) => {

```

```

await queryInterface.addColumn('Products', 'scheduleId', {
  type: Sequelize.INTEGER,
  allowNull: true,
  references: {
    model: 'Schedules',
    key: 'id'
  },
  onDelete: 'SET NULL'
})
},

down: async (queryInterface, Sequelize) => {
  await queryInterface.removeColumn('Products', 'scheduleId')
}
}

```

10.3.3. Modelo - Schedule.js

```

'use strict'
const { Model } = require('sequelize')

module.exports = (sequelize, DataTypes) => {
  class Schedule extends Model {
    static associate (models) {
      Schedule.belongsTo(models.Restaurant, {
        foreignKey: 'restaurantId',
        as: 'restaurant'
      })
    }

```

```

      Schedule.hasMany(models.Product, {
        foreignKey: 'scheduleId',
        as: 'products'
      })
    }
  }

```

```

}

```

```

Schedule.init({
  id: {
    type: DataTypes.INTEGER,

```

```

    autoIncrement: true,
    primaryKey: true
  },
  startTime: {
    type: DataTypes.TIME,
    allowNull: false
  },
  endTime: {
    type: DataTypes.TIME,
    allowNull: false
  },
  restaurantId: {
    type: DataTypes.INTEGER,
    allowNull: false
  }
}, {
  sequelize,
  modelName: 'Schedule',
  tableName: 'Schedules'
})

return Schedule
}

```

10.3.4. Modelo - Product.js (Modificaciones)

// Añadir en Product.js

```

static associate (models) {
// ... asociaciones existentes ...

Product.belongsTo(models.Schedule, {
  foreignKey: 'scheduleId',
  as: 'schedule'
})
}

```

// Añadir campo scheduleId en init

```

Product.init({
// ... campos existentes ...

```

```

scheduleId: {
  type: DataTypes.INTEGER,
  allowNull: true
}, {
  sequelize,
  modelName: 'Product'
})

```

10.3.5. Rutas - ScheduleRoutes.js

```

'use strict'
const ScheduleController = require('../controllers/ScheduleController')
const { isLoggedIn, hasRole } =
  require('../middlewares/AuthMiddleware')
const { checkRestaurantOwnership } =
  require('../middlewares/RestaurantMiddleware')
const { handleValidation } =
  require('../middlewares/ValidationHandlingMiddleware')
const { create, update } =
  require('../controllers/validation/ScheduleValidation')
const { checkEntityExists } =
  require('../middlewares/EntityMiddleware')
const { Restaurant, Schedule } = require('../models')

module.exports = (options) => {
  const app = options.app

  // GET /restaurants/:restaurantId/schedules - Listar schedules
  // POST /restaurants/:restaurantId/schedules - Crear schedule
  app.route('/restaurants/:restaurantId/schedules')
    .get(
      checkEntityExists(Restaurant, 'restaurantId'),
      ScheduleController.index
    )
    .post(
      isLoggedIn,
      hasRole('owner'),
      checkEntityExists(Restaurant, 'restaurantId'),
      checkRestaurantOwnership,
      create,

```

```

    handleValidation,
    ScheduleController.create
  )

  // PUT /restaurants/:restaurantId/schedules/:scheduleId
  // DELETE /restaurants/:restaurantId/schedules/:scheduleId
  app.route('/restaurants/:restaurantId/schedules/:scheduleId')
    .put(
      isLoggedIn,
      hasRole('owner'),
      checkEntityExists(Restaurant, 'restaurantId'),
      checkEntityExists(Schedule, 'scheduleId'),
      checkRestaurantOwnership,
      update,
      handleValidation,
      ScheduleController.update
    )
    .delete(
      isLoggedIn,
      hasRole('owner'),
      checkEntityExists(Restaurant, 'restaurantId'),
      checkEntityExists(Schedule, 'scheduleId'),
      checkRestaurantOwnership,
      ScheduleController.destroy
    )
  }

```

10.3.6. Validaciones - ScheduleValidation.js

```

'use strict'
const { check } = require('express-validator')

// Validar formato de hora (HH:mm:ss)
const validateTimeFormat = (value) => {
  const timeRegex = /^([01]\d|2[0-3]):([0-5]\d):([0-5]\d)$/
  if (!timeRegex.test(value)) {
    throw new Error('Time must be in HH:mm:ss format')
  }
  return true
}

```

```

// Validar que endTime es posterior a startTime
const validateEndTimeAfterStartTime = (endTime, { req }) => {
  if (endTime <= req.body.startTime) {
    throw new Error('End time must be after start time')
  }
  return true
}

const create = [
  check('startTime')
  .exists().withMessage('Start time is required')
  .custom(validateTimeFormat),

  check('endTime')
  .exists().withMessage('End time is required')
  .custom(validateTimeFormat)
  .custom(validateEndTimeAfterStartTime)
]

const update = [
  check('startTime')
  .exists().withMessage('Start time is required')
  .custom(validateTimeFormat),

  check('endTime')
  .exists().withMessage('End time is required')
  .custom(validateTimeFormat)
  .custom(validateEndTimeAfterStartTime)
]

module.exports = {
  create,
  update
}

```

10.3.7. Controlador - ScheduleController.js

```

'use strict'
const { Schedule } = require('../models')

// Listar schedules de un restaurante
const index = async (req, res) => {

```



```
try {
  const schedules = await Schedule.findAll({
    where: { restaurantId: req.params.restaurantId },
    order: [['startTime', 'ASC']]
  })
  res.json(schedules)
} catch (err) {
  res.status(500).send(err)
}
}
```

// Crear schedule

```
const create = async (req, res) => {
  try {
    const newSchedule = await Schedule.create({
      startTime: req.body.startTime,
      endTime: req.body.endTime,
      restaurantId: req.params.restaurantId
    })
    res.json(newSchedule)
  } catch (err) {
    res.status(500).send(err)
  }
}
```

// Actualizar schedule

```
const update = async (req, res) => {
  try {
    await Schedule.update(
      {
        startTime: req.body.startTime,
        endTime: req.body.endTime
      },
      {
        where: { id: req.params.scheduleId }
      }
    )
    const updatedSchedule = await
    Schedule.findByPk(req.params.scheduleId)
    res.json(updatedSchedule)
  }
}
```

```

    } catch (err) {
      res.status(500).send(err)
    }
  }

  // Eliminar schedule
  const destroy = async (req, res) => {
    try {
      const result = await Schedule.destroy({
        where: { id: req.params.scheduleId }
      })
      res.json(result)
    } catch (err) {
      res.status(500).send(err)
    }
  }

  module.exports = {
    index,
    create,
    update,
    destroy
  }

```

10.3.8. Validación - ProductValidation.js (Modificaciones)

```

const { check } = require('express-validator')
const { Product, Schedule } = require('../models')

// Validación para creación de producto
const checkScheduleBelongsToRestaurantOnCreate =
  check('scheduleId')
    .optional()
    .custom(async (value, { req }) => {
      if (!value) return true

```

```

      const schedule = await Schedule.findOne({
        where: {
          id: value,
          restaurantId: req.body.restaurantId

```

```
    }  
  })  
  
  if (!schedule) {  
    throw new Error('Schedule does not belong to restaurant')  
  }  
  return true  
}
```

}}

```
// Validación para edición de producto  
const checkScheduleBelongsToRestaurantOnUpdate =  
check('scheduleId')  
.optional()  
.custom(async (value, { req }) => {  
  if (!value) return true
```

```
  // Obtener el producto para saber su restaurante  
  const product = await Product.findByPk(req.params.productId)  
  
  const schedule = await Schedule.findOne({  
    where: {  
      id: value,  
      restaurantId: product.restaurantId  
    }  
  })  
  
  if (!schedule) {  
    throw new Error('Schedule does not belong to product restaurant')  
  }  
  return true  
}
```

}}

```
// Añadir a las validaciones existentes  
const create = [  
  // ... validaciones existentes ...
```

```
checkScheduleBelongsToRestaurantOnCreate
]
```

```
const update = [
// ... validaciones existentes ...
checkScheduleBelongsToRestaurantOnUpdate
]
```

10.3.9. Controlador - RestaurantController.js (showWithActiveProducts)

```
// Mostrar restaurante con solo productos activos
const showWithActiveProducts = async (req, res) => {
try {
const { Op } = require('sequelize')
```

```
// Obtener hora actual en formato HH:mm:ss
const currentTime = new Date().toString().split(' ')[0]

// Buscar restaurante con productos activos
const restaurant = await Restaurant.findByPk(req.params.restaurantId, {
  include: [
    {
      model: Product,
      as: 'products',
      include: [
        {
          model: Schedule,
          as: 'schedule',
          where: {
            startTime: { [Op.lte]: currentTime },
            endTime: { [Op.gte]: currentTime }
          },
          required: true // INNER JOIN - solo productos con schedule válido
        }
      ]
    }
  ]
})
})
```

```
if (!restaurant) {
  return res.status(404).send('Restaurant not found')
}

res.json(restaurant)

} catch (err) {
res.status(500).send(err)
}
}

module.exports = {
// ... otras funciones ...
showWithActiveProducts
}
```

11. Checklist para el Examen

Antes del Examen

- ☐ Revisar esta guía completa
- ☐ Practicar con los ejemplos resueltos
- ☐ Entender el flujo: Migración → Modelo → Rutas → Validaciones → Controlador → Middlewares
- ☐ Repasar conceptos de JavaScript (async/await, arrow functions, destructuring)
- ☐ Verificar que Node.js está instalado (node --version)
- ☐ Verificar que npm está instalado (npm --version)

Durante el Examen

Paso 0: Configuración Inicial

- ☐ Descomprimir el proyecto
- ☐ Abrir terminal en la carpeta del proyecto
- ☐ Ejecutar npm run install:all:win (Windows) o npm run install:all:bash (Linux/Mac)
- ☐ Verificar que no hay errores de instalación

Paso 1: Migraciones (1-2 puntos)

- ☐ Localizar archivo de migración en `src/database/migrations/`
- ☐ Crear tabla con `createTable` o modificar con `addColumn`
- ☐ Definir todas las columnas necesarias (id, campos de datos, FKs)
- ☐ Añadir claves foráneas con `references` y `onDelete`
- ☐ Incluir `createdAt` y `updatedAt`
- ☐ Implementar función `down` para deshacer cambios
- ☐ Si se modifica tabla existente, crear migración adicional

Paso 2: Modelos (1-2 puntos)

- ☐ Localizar o crear archivo en `src/models/`
- ☐ Definir clase que extiende `Model`
- ☐ Implementar método `associate` con relaciones
- ☐ Usar `belongsTo` para FK, `hasMany` para inversa
- ☐ Definir todas las columnas en `init`
- ☐ Añadir validaciones si es necesario
- ☐ Si hay atributos virtuales, implementar `getter` y función de cálculo
- ☐ Modificar modelos relacionados (añadir `hasMany`, `hasOne`, etc.)

Paso 3: Rutas (1-2 puntos)

- ☐ Localizar o crear archivo en `src/routes/`
- ☐ Importar controlador, middlewares, validaciones y modelos
- ☐ Definir rutas con `app.route()`
- ☐ Usar método HTTP correcto (GET, POST, PUT, PATCH, DELETE)
- ☐ Añadir middlewares en orden correcto:
 1. Autenticación (`isLoggedIn`)
 2. Rol (`hasRole`)
 3. Existencia de entidades (`checkEntityExists`)
 4. Validaciones
 5. Manejo de validaciones (`handleValidation`)
 6. Middlewares de negocio
 7. Controlador
- ☐ Verificar nombres de parámetros en rutas (`:restaurantId`, `:reviewId`)

Paso 4: Validaciones (1-2 puntos)

- [] Localizar o crear archivo en `src/controllers/validation/`
- [] Importar check de `express-validator`
- [] Crear array `create` con validaciones de creación
- [] Crear array `update` con validaciones de edición
- [] Para cada campo obligatorio: `exists()`, tipo apropiado, `withMessage()`
- [] Para campos opcionales: `optional()`
- [] Añadir validaciones específicas (rangos, formatos, etc.)
- [] Si hay validaciones personalizadas, usar `.custom()`
- [] Exportar validaciones con `module.exports`

Paso 5: Controlador (2 puntos)

- [] Localizar o crear archivo en `src/controllers/`
- [] Importar modelos necesarios
- [] Implementar función `index` (GET - listar)
 - [] Usar `findAll` con `where`
 - [] Añadir `order` si es necesario
- [] Implementar función `create` (POST - crear)
 - [] Usar `create` con datos de `req.body`
 - [] Añadir `userId` o `restaurantId` desde `req.user` o `req.params`
 - [] Manejar lógica especial (ej: primera dirección = default)
- [] Implementar función `update` (PUT - actualizar)
 - [] Usar `update` con `where`
 - [] Devolver entidad actualizada con `findByPk`
- [] Implementar función `destroy` (DELETE - eliminar)
 - [] Usar `destroy` con `where`
- [] Implementar funciones especiales si las hay (ej: `setDefault`, `showWithActiveProducts`)
- [] Envolver todo en `try-catch`
- [] Exportar funciones con `module.exports`

Paso 6: Middlewares (2 puntos)

- [] Localizar o crear archivo en `src/middlewares/`
- [] Implementar verificación de propiedad (ownership)
 - [] Buscar entidad con `findByPk`
 - [] Verificar que `entity.userId === req.user.id`

- ☐ Devolver 403 si no es dueño
- ☐ Implementar verificaciones de lógica de negocio
 - ☐ Consultar base de datos según requisito
 - ☐ Devolver 409 si no se cumple condición
- ☐ Implementar verificaciones de pertenencia (belongsTo)
 - ☐ Verificar que FK coincide con entidad padre
 - ☐ Devolver 422 si no pertenece
- ☐ Usar next() si todo está bien
- ☐ Envolver en try-catch
- ☐ Exportar middlewares con `module.exports`

Paso 7: Pruebas

- ☐ Ejecutar migraciones: `npm run migrate:backend`
- ☐ Si hay errores de migración, revisar sintaxis
- ☐ Ejecutar backend: `npm run start:backend`
- ☐ Si hay errores de inicio, revisar imports y sintaxis
- ☐ Ejecutar tests: `npm run test:backend`
- ☐ Leer errores de tests y corregir
- ☐ Repetir hasta que todos los tests pasen

Paso 8: Entrega

- ☐ Detener backend (Ctrl+C)
- ☐ Borrar carpeta `node_modules` de backend
- ☐ Crear ZIP con todo el proyecto
- ☐ Verificar que el ZIP incluye tus cambios
- ☐ Avisar al profesor
- ☐ Subir a plataforma cuando profesor dé OK
- ☐ Esperar a que aparezca enlace al ZIP
- ☐ Descargar ZIP para verificar
- ☐ Enviar examen

12. Errores Comunes y Soluciones

12.1. Errores de Migración

Error: ERROR: relation "Restaurants" does not exist

- **Causa:** Tabla referenciada no existe

- **Solución:** Ejecutar migraciones en orden correcto, crear tabla padre primero

Error: column "createdAt" does not exist

- **Causa:** Falta añadir createdAt/updatedAt
- **Solución:** Siempre incluir ambas columnas en createTable

Error: type "DATETIME" does not exist

- **Causa:** Tipo de dato incorrecto (MySQL vs PostgreSQL)
- **Solución:** Usar Sequelize.DATE en lugar de DATETIME

12.2. Errores de Modelo

Error: Associate is not a function

- **Causa:** Método associate mal escrito (ej: asociate)
- **Solución:** Escribir exactamente static associate (models)

Error: Cannot read property 'foreignKey' of undefined

- **Causa:** FK no definida en migración o modelo
- **Solución:** Verificar que FK existe en ambos

Error: Model.hasMany called with something that's not a subclass of Sequelize.Model

- **Causa:** Modelo no importado o no existe
- **Solución:** Verificar imports y que modelo está definido

12.3. Errores de Rutas

Error: Cannot GET /restaurants/1/reviews

- **Causa:** Ruta no definida o mal escrita
- **Solución:** Verificar que ruta está en archivo de rutas y coincide exactamente

Error: ReviewController.index is not a function

- **Causa:** Función no exportada o mal escrita
- **Solución:** Verificar module.exports = { index, ... }

Error: Test falla con 401 Unauthorized

- **Causa:** Falta middleware isLoggedIn
- **Solución:** Añadir isLoggedIn antes del controlador

Error: Test falla con 403 Forbidden

- **Causa:** Falta middleware hasRole
- **Solución:** Añadir hasRole('customer') o hasRole('owner')

12.4. Errores de Validación

Error: validationResult is not a function

- **Causa:** No se importó correctamente
- **Solución:** Verificar `const { check } = require('express-validator')`

Error: Test falla con 422 Unprocessable Entity

- **Causa:** Validación falla
- **Solución:** Revisar qué campo falla en logs y corregir validación

Error: Cannot read property 'body' of undefined

- **Causa:** En validación personalizada, req no disponible
- **Solución:** Usar segundo parámetro: `custom((value, { req }) => ...)`

12.5. Errores de Controlador

Error: await is only valid in async function

- **Causa:** Función no es async
- **Solución:** Añadir async antes de parámetros: `const index = async (req, res) => {}`

Error: Cannot read property 'id' of undefined

- **Causa:** req.user o req.params no existen
- **Solución:** Verificar que middleware de autenticación está antes

Error: findAll is not a function

- **Causa:** Modelo no importado correctamente
- **Solución:** Verificar `const { Review } = require('../models')`

Error: Test falla con 500 Internal Server Error

- **Causa:** Error en lógica del controlador
- **Solución:** Revisar logs de consola, probablemente error de sintaxis o consulta

12.6. Errores de Middleware

Error: next is not defined

- **Causa:** No se incluyó parámetro next
- **Solución:** Añadir tercer parámetro: (req, res, next) => {}

Error: Cannot set headers after they are sent

- **Causa:** Se envía respuesta y luego se llama a next()
- **Solución:** Usar return res.status(...).send(...) sin llamar a next() después

Error: Test se queda esperando timeout

- **Causa:** Middleware no llama a next() ni envía respuesta
- **Solución:** Verificar que todas las rutas terminan con next() o res.send()

12.7. Errores de JavaScript

Error: SyntaxError: Unexpected token

- **Causa:** Error de sintaxis (falta llave, paréntesis, etc.)
- **Solución:** Revisar línea indicada, contar llaves/paréntesis

Error: ReferenceError: X is not defined

- **Causa:** Variable no declarada o no importada
- **Solución:** Declarar con const o importar módulo

Error: TypeError: X is not a function

- **Causa:** Intentar llamar algo que no es función
 - **Solución:** Verificar que es función y está bien exportada/importada
-

Apéndice A: Comandos npm Útiles

Instalar dependencias

npm run install:all:win # Windows
npm run install:all:bash # Linux/Mac

Migraciones

npm run migrate:backend # Ejecutar migraciones
npm run migrate:backend:undo # Deshacer última migración

Ejecutar backend

npm run start:backend # Modo normal

Depuración: F5 en VSCode con "Debug Backend"

Tests

npm run test:backend # Ejecutar todos los tests
npm run test:backend -- --testNamePattern="reviews" # Test específico

Limpiar y reiniciar

rm -rf node_modules # Borrar dependencias (Linux/Mac)
rmdir /s node_modules # Borrar dependencias (Windows)

Apéndice B: Atajos de VSCode

Ctrl+Space - Autocompletado
Ctrl+Click - Ir a definición
F2 - Renombrar símbolo
Ctrl+/ - Comentar/descomentar línea
Ctrl+Shift+K - Borrar línea
Alt+↑/↓ - Mover línea arriba/abajo
Ctrl+D - Seleccionar siguiente ocurrencia
Ctrl+F - Buscar en archivo
Ctrl+Shift+F - Buscar en proyecto
F5 - Iniciar depuración
F10 - Paso a paso sobre (debug)
F11 - Paso a paso dentro (debug)

Apéndice C: Códigos de Estado HTTP

200 OK - Operación exitosa
201 Created - Recurso creado
401 Unauthorized - No autenticado
403 Forbidden - No autorizado (autenticado pero sin permisos)
404 Not Found - Recurso no existe
409 Conflict - Conflicto de lógica de negocio
422 Unprocessable Entity - Validación falla
500 Internal Server Error - Error del servidor

Apéndice D: Estructura de req y res

// req (Request object)
req.params // Parámetros de ruta: /restaurants/:id → [req.params.id](#)
req.query // Query string: /reviews?stars=5 → req.query.stars
req.body // Datos enviados en POST/PUT
req.user // Usuario autenticado (añadido por middleware)
[req.user.id](#) // ID del usuario
req.user.userId // Rol: 'customer' o 'owner'

// res (Response object)
res.json(data) // Enviar JSON
res.status(404).send(msg) // Enviar con código de estado
res.send(msg) // Enviar texto

Consejos Finales

1. **Lee el enunciado completo** antes de empezar
2. **Sigue el orden:** Migración → Modelo → Rutas → Validaciones → Controlador → Middlewares
3. **Copia la estructura** de ejemplos similares (Review, ShippingAddress, Schedule)
4. **Prueba frecuentemente** con `npm run test:backend`
5. **Lee los mensajes de error** con atención, suelen indicar exactamente qué falta
6. **Verifica nombres** (mayúsculas, plural/singular, parámetros de rutas)
7. **No modifiques los tests** - están para ayudarte
8. **Gestiona el tiempo:** 1.5-2 horas es suficiente si sigues la estructura
9. **No te bloques:** Si algo no funciona, sigue con el siguiente apartado
10. **Revisa antes de entregar:** ¿Pasaron todos los tests? ¿Borraste `node_modules`?

¡Mucha suerte en el examen! ☺